Design patterns in Go

Contents

1. What Go changes about patterns
2. Functional options
3. Constructor / factory functions
4. Builder
5. Strategy: interface-based
6. Decorator (wrap an interface)
7. Middleware (decorator for handlers)
8. Adapter
9. Facade
10. Proxy
11. Observer (via channels)
12. Iterator (Go 1.23+)
13. State machine
14. Singleton (rare; use sync.Once)
15. Object pool (sync.Pool)
16. Repository
17. Dependency injection (constructor)
18. Worker pool
19. Pipeline
20. Fan-in / fan-out
21. Pub/sub with channels
22. Chain of responsibility
23. Embedding (Go's composition)
24. What Go doesn't need (and you shouldn't write)
25. Pattern selection guide
26. Best practices
27. Common gotchas
28. Quick reference

The patterns that show up in real Go code. Some are classic GoF adapted to Go; many are Go-specific because the language picks different defaults (composition over inheritance, interfaces over class hierarchies, channels over shared state).

What Go changes about patterns

Go's design removes the need for several classic patterns and reshapes others.

Classic pattern	What Go does instead
Abstract Factory	Plain factory function, returning an interface
Class hierarchy	Composition via embedded structs
Inheritance	Embedding + interface satisfaction
Template Method	Function values or interfaces passed in
Visitor	Type switches or interface methods
Singleton	`sync.Once` + package-level var (and rarely needed)
Builder (long chain)	Functional options
Observer	Channels
Iterator	`range`, `range func` (Go 1.23+)

So when you read about "Java design patterns," half of them either don't apply or have a one-line Go equivalent.

1. Functional options

The most idiomatic Go pattern. Used by `grpc`, `redis-go`, `http`, almost every modern Go library.

The problem: you have a constructor with many optional parameters. Variadic args don't help (order matters), config structs are clunky (every field gets a zero value when omitted).

The solution: each option is a function that modifies the target.

```go
type Server struct {
    addr    string
    timeout time.Duration
    logger  *slog.Logger
    tls     *tls.Config
}

type Option func(*Server)

func WithTimeout(d time.Duration) Option {
    return func(s *Server) { s.timeout = d }
}

func WithLogger(l *slog.Logger) Option {
    return func(s *Server) { s.logger = l }
}

func WithTLS(cfg *tls.Config) Option {
    return func(s *Server) { s.tls = cfg }
}

func NewServer(addr string, opts ...Option) *Server {
    s := &Server{
        addr:    addr,
        timeout: 30 * time.Second,        // defaults
        logger:  slog.Default(),
    }
    for _, opt := range opts {
        opt(s)
    }
    return s
}
```

Usage:

```go
srv := NewServer(":8080",
    WithTimeout(time.Minute),
    WithTLS(tlsCfg),
)
```

Why it wins:

- Required args stay positional (`addr`).
- Each option is independently named and documented.
- Adding new options doesn't break existing callers.
- Defaults are visible in the constructor.
- Order of options doesn't matter.

Variants:

- Return error from options: `type Option func(*Server) error`. Useful when an option can be invalid.
- Use an interface: `type Option interface { apply(*Server) }`. Helps with grouping and identity but adds boilerplate.

2. Constructor / factory functions

Go has no constructors. Convention is a `New<Type>` function.

```
type Cache struct { ... }

func NewCache(cap int) *Cache {
    return &Cache{
        items: make(map[string]any, cap),
        cap:   cap,
    }
}
```

Variants:

- `New(...)` (no type suffix) for the primary constructor of a package. Returns the package's main type.
- `NewFromX(...)` when constructing from a different source.
- Multiple constructors are fine: `NewLRU`, `NewFIFO`, both returning a `*Cache`.

If the type implements an interface that callers consume, return the concrete type from the constructor. The caller can assign it to the interface.

```
// good: caller decides what they want
func NewMemoryStore() *MemoryStore { ... }

// not as good: forces interface, harder to use methods unique to MemoryStore
func NewMemoryStore() Store { ... }
```

3. Builder

Useful when:

- Construction has many steps that must run in order.
- Validation happens at the end.
- Some fields depend on prior ones.

```go
type RequestBuilder struct {
    req *http.Request
    err error
}

func NewRequest(method, url string) *RequestBuilder {
    req, err := http.NewRequest(method, url, nil)
    return &RequestBuilder{req: req, err: err}
}

func (b *RequestBuilder) Header(k, v string) *RequestBuilder {
    if b.err == nil { b.req.Header.Set(k, v) }
    return b
}

func (b *RequestBuilder) Body(r io.Reader) *RequestBuilder {
    if b.err == nil { b.req.Body = io.NopCloser(r) }
    return b
}

func (b *RequestBuilder) Build() (*http.Request, error) {
    return b.req, b.err
}
```

Usage:

```go
req, err := NewRequest("POST", url).
    Header("Content-Type", "application/json").
    Body(body).
    Build()
```

The error-on-the-builder pattern is the Go take: collect errors during build, surface them all at the end. No need to handle each step.

For simple cases, use functional options. Reach for builder when steps are ordered or stateful.

4. Strategy: interface-based

Define an interface for the algorithm. Pass implementations.

```go
type Hasher interface {
    Hash([]byte) []byte
}

type SHA256Hasher struct{}
func (SHA256Hasher) Hash(b []byte) []byte { ... }

type BLAKE2Hasher struct{}
func (BLAKE2Hasher) Hash(b []byte) []byte { ... }

func Sign(payload []byte, h Hasher) []byte {
    return h.Hash(payload)
}
```

Or, more idiomatic for one-method strategies, pass a function:

```go
func Sign(payload []byte, hash func([]byte) []byte) []byte {
    return hash(payload)
}

Sign(data, sha256.Sum256... wrap as needed)
```

The function form wins when:

- There's exactly one method.
- The "strategy" is a single decision, not an object with state.

The interface form wins when:

- Multiple methods belong together.
- Implementations need their own state.
- You want named types for clarity (`MD5Hasher`, `BLAKE2Hasher`).

5. Decorator (wrap an interface)

Take an interface, return one that adds behavior around it.

```go
type Storage interface {
    Get(key string) ([]byte, error)
    Put(key string, value []byte) error
}

func WithLogging(s Storage, l *slog.Logger) Storage {
    return &loggingStorage{inner: s, log: l}
}

type loggingStorage struct {
    inner Storage
    log   *slog.Logger
}

func (s *loggingStorage) Get(key string) ([]byte, error) {
    s.log.Info("get", "key", key)
    return s.inner.Get(key)
}

func (s *loggingStorage) Put(key string, value []byte) error {
    s.log.Info("put", "key", key)
    return s.inner.Put(key, value)
}

// usage
storage := WithLogging(WithMetrics(redisStore), logger)
```

Chain decorators by passing the result back in. Each decorator adds one orthogonal concern: logging, metrics, retries, caching, circuit breakers.

The HTTP middleware pattern is decorator applied to `http.Handler` (next section).

6. Middleware (decorator for handlers)

HTTP middleware is the most common decorator in Go.

```go
type Middleware func(http.Handler) http.Handler

func Logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        slog.Info("request", "path", r.URL.Path, "dur", time.Since(start))
    })
}

func Auth(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        token := r.Header.Get("Authorization")
        if !valid(token) {
            http.Error(w, "unauthorized", 401)
            return
        }
        next.ServeHTTP(w, r)
    })
}

// chain helper
func Chain(h http.Handler, ms ...Middleware) http.Handler {
    for i := len(ms) - 1; i >= 0; i-- {
        h = ms[i](h)
    }
    return h
}

http.Handle("/api/", Chain(apiHandler, Logging, Auth, RateLimit))
```

The same shape applies to gRPC interceptors, database query wrappers, event bus middleware. Anywhere there's "do something before/after a call, possibly skipping it," middleware fits.

7. Adapter

Convert one interface to another. Common when integrating libraries with mismatched APIs.

```go
// you have:
type Old interface { DoOldThing(s string) error }

// you need:
type New interface { Execute(ctx context.Context, in any) error }

// adapter:
type OldToNew struct { old Old }

func (a *OldToNew) Execute(ctx context.Context, in any) error {
    s, ok := in.(string)
    if !ok { return fmt.Errorf("expected string, got %T", in) }
    return a.old.DoOldThing(s)
}
```

In Go, this often takes the form of small functions or anonymous types. The `http.HandlerFunc` type is exactly an adapter: it converts a function into an `http.Handler`.

```go
type HandlerFunc func(ResponseWriter, *Request)
func (f HandlerFunc) ServeHTTP(w ResponseWriter, r *Request) { f(w, r) }
```

8. Facade

A simpler interface over a complex subsystem. Just a struct that hides the complexity.

```go
// internal: many packages, many calls
package orders

type Service struct {
    db        *sql.DB
    inventory *inventory.Client
    payment   *payment.Client
    notify    *notification.Bus
}

func (s *Service) Checkout(ctx context.Context, cart Cart) (Receipt, error) {
    // 1. reserve inventory
    // 2. charge card
    // 3. create order row
    // 4. send notification
    // ...
}
```

The caller writes `orders.Checkout(...)`; doesn't know about inventory, payment, notification. That's facade. It's just naming things one level up.

9. Proxy

A stand-in for a real object. Same interface, but the proxy intercepts calls (caching, lazy loading, remote calls).

```
type CachingUserStore struct {
    inner UserStore
    cache *lru.Cache[string, *User]
}

func (s *CachingUserStore) Get(id string) (*User, error) {
    if u, ok := s.cache.Get(id); ok { return u, nil }
    u, err := s.inner.Get(id)
    if err == nil { s.cache.Add(id, u) }
    return u, err
}
```

Mechanically identical to decorator. The labels differ by intent: decorator adds behavior, proxy stands in for the real thing. In Go code, the distinction usually doesn't matter.

10. Observer (via channels)

Subscribers receive events.

```
type EventBus[T any] struct {
    mu   sync.RWMutex
    subs []chan T
}

func (b *EventBus[T]) Subscribe() <-chan T {
    ch := make(chan T, 16)
    b.mu.Lock()
    b.subs = append(b.subs, ch)
    b.mu.Unlock()
    return ch
}

func (b *EventBus[T]) Publish(event T) {
    b.mu.RLock()
    defer b.mu.RUnlock()
    for _, ch := range b.subs {
        select {
        case ch <- event:
        default:                    // drop if subscriber is slow
        }
    }
}
```

Choices to make:

- Drop or block when a subscriber is slow. Drop is common; block creates head-of-line blocking.

- Cleanup of dead subscribers. Add `Unsubscribe` or detect closed channels.
- Synchronous vs async dispatch. A goroutine per `Publish` call decouples publishers from slow subscribers.

For cross-process observer, use a real message broker (NATS, Kafka, Redis pub/sub).

11. Iterator (Go 1.23+)

Range over function lets a type expose iteration without exposing internals.

```
type Tree struct { root *node }

func (t *Tree) InOrder() iter.Seq[int] {
    return func(yield func(int) bool) {
        var walk func(n *node) bool
        walk = func(n *node) bool {
            if n == nil { return true }
            if !walk(n.left) { return false }
            if !yield(n.value) { return false }
            if !walk(n.right) { return false }
            return true
        }
        walk(t.root)
    }
}

for v := range tree.InOrder() {
    fmt.Println(v)
}
```

For collections, `iter.Seq[T]` (one value) and `iter.Seq2[K, V]` (two values, like map iteration) are the standard return types.

Before Go 1.23: callback-based iteration:

```
func (t *Tree) Visit(f func(int) bool) { ... }
tree.Visit(func(v int) bool { ... return true })
```

For new code, prefer `iter.Seq` if you're on 1.23+.

12. State machine

Either explicit (each state is a value), or via function values.

Explicit state, switch dispatch

```go
type State int

const (
    Pending State = iota
    Active
    Done
    Failed
)

type Order struct {
    state State
}

func (o *Order) Activate() error {
    if o.state != Pending { return errors.New("not pending") }
    o.state = Active
    return nil
}

func (o *Order) Complete() error {
    if o.state != Active { return errors.New("not active") }
    o.state = Done
    return nil
}
```

State as a function value

```go
type stateFn func(*Order) stateFn

func pending(o *Order) stateFn {
    // do pending work
    return active
}

func active(o *Order) stateFn { ... }
func done(o *Order) stateFn   { return nil }
```

Useful for lexers, parsers, complex protocols (the standard library's text scanner uses this).

For most business state machines, the explicit `State` constant with method dispatch is clearer.

13. Singleton (rare; use sync.Once)

A single instance with lazy initialization, thread-safe.

```go
var (
    once     sync.Once
    instance *DB
)

func GetDB() *DB {
    once.Do(func() {
        instance = openDB()
    })
    return instance
}
```

Reasons singletons are rare in Go:

- Package-level vars work for true globals (loggers, default clients).
- Most “singletons” are better as explicit dependencies passed in.
- Tests want to swap the instance; a true singleton fights that.

Use when:

- The thing is genuinely process-global (a TLS root pool, a config loaded from env once).
- Construction is expensive and shared across many callers.

Don’t use when you’re tempted because “passing it everywhere is annoying.” That’s the wrong fix.

14. Object pool (sync.Pool)

Reusable allocations for hot paths.

```go
var bufPool = sync.Pool{
    New: func() any { return new(bytes.Buffer) },
}

func use() {
    buf := bufPool.Get().(*bytes.Buffer)
    defer bufPool.Put(buf)
    buf.Reset()
    // ... use buf
}
```

Notes:

- Items may be evicted on GC. Never assume the same object comes back.
- Always reset state on `Get` or on `Put`. Either is fine; pick one and stick to it.
- Don’t pool things that hold large memory (the GC will eat them anyway, and they leak between cycles).
- Use only when allocation profiling shows pressure. Premature pooling adds complexity.

15. Repository

Hide data-access details behind an interface so the business layer doesn't care about the storage.

```go
// domain
type UserRepository interface {
    Get(ctx context.Context, id string) (*User, error)
    Save(ctx context.Context, u *User) error
    List(ctx context.Context, filter Filter) ([]*User, error)
}

// implementation
type PostgresUserRepo struct { db *sql.DB }

func (r *PostgresUserRepo) Get(ctx context.Context, id string) (*User, error) { ... }
func (r *PostgresUserRepo) Save(ctx context.Context, u *User) error           { ... }
func (r *PostgresUserRepo) List(ctx context.Context, f Filter) ([]*User, error) {
    ...
}

// service uses interface
type UserService struct { repo UserRepository }
```

Benefits:

- Testable: pass a fake `UserRepository` in tests.
- Swappable: switch from Postgres to MongoDB without changing business logic.
- Boundaries: explicit storage contract.

Costs:

- Indirection.
- Sometimes the interface becomes an exact copy of the DB API, which is over-engineering.

> **Rule of thumb**
>
> Define the repository interface from the caller's perspective. If the business layer only needs `GetActive(ctx) ([]*User, error)`, that's all the interface should have, even if the DB has 20 query options.

16. Dependency injection (constructor)

Go's idiomatic DI is plain constructor injection. No frameworks needed.

```go
type Service struct {
    repo   UserRepository
    cache  Cache
    logger *slog.Logger
}

func NewService(repo UserRepository, cache Cache, logger *slog.Logger) *Service {
    return &Service{repo: repo, cache: cache, logger: logger}
}

// in main
db := setupDB()
repo := NewPostgresUserRepo(db)
cache := lru.New[string, *User](1000)
svc := NewService(repo, cache, slog.Default())
```

For more complex graphs, frameworks exist:

- `wire` (google/wire): compile-time DI via code generation. Catches missing dependencies at build time.
- `fx` (uber-go/fx): runtime DI with lifecycle hooks. More magical, more tradeoffs.

For most services, hand-wired constructors in `main` are clearer than a DI framework.

17. Worker pool

Bounded concurrency for a stream of jobs.

```go
func WorkerPool[T any, R any](
    ctx context.Context,
    jobs <-chan T,
    n int,
    work func(context.Context, T) (R, error),
) <-chan R {
    results := make(chan R, n)
    var wg sync.WaitGroup
    wg.Add(n)
    for i := 0; i < n; i++ {
        go func() {
            defer wg.Done()
            for j := range jobs {
                if r, err := work(ctx, j); err == nil {
                    select {
                    case results <- r:
                    case <-ctx.Done(): return
                    }
                }
            }
        }()
    }
    go func() { wg.Wait(); close(results) }()
    return results
}
```

Variations:

- Use `errgroup.Group` if you want first-error cancellation.
- Add `SetLimit(n)` on `errgroup` (Go 1.20+) for the bounded concurrency built-in.

18. Pipeline

Chain of stages, each in its own goroutine, connected by channels.

```go
func gen(nums ...int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for _, n := range nums { out <- n }
    }()
    return out
}

func square(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for n := range in { out <- n * n }
    }()
    return out
}

for v := range square(square(gen(1, 2, 3))) {
    fmt.Println(v)         // 1, 16, 81
}
```

Each stage closes its output when its input is drained. Cancellation flows by closing the source. For real pipelines, add `ctx` and `select` for clean shutdown.

19. Fan-in / fan-out

Fan-out: distribute work across multiple goroutines. Fan-in: combine multiple channels into one.

```go
func fanIn[T any](chans ...<-chan T) <-chan T {
    out := make(chan T)
    var wg sync.WaitGroup
    wg.Add(len(chans))
    for _, c := range chans {
        go func(c <-chan T) {
            defer wg.Done()
            for v := range c { out <- v }
        }(c)
    }
    go func() { wg.Wait(); close(out) }()
    return out
}
```

The reverse (fan-out) is “spawn N workers reading from the same channel.” It’s the worker-pool pattern by another name.

20. Pub/sub with channels

A simple in-process pub/sub:

```go
type Topic[T any] struct {
    mu   sync.RWMutex
    subs map[string]chan T
}

func New[T any]() *Topic[T] {
    return &Topic[T]{subs: map[string]chan T{}}
}

func (t *Topic[T]) Subscribe(id string) <-chan T {
    t.mu.Lock()
    defer t.mu.Unlock()
    ch := make(chan T, 16)
    t.subs[id] = ch
    return ch
}

func (t *Topic[T]) Unsubscribe(id string) {
    t.mu.Lock()
    defer t.mu.Unlock()
    if ch, ok := t.subs[id]; ok {
        close(ch)
        delete(t.subs, id)
    }
}

func (t *Topic[T]) Publish(v T) {
    t.mu.RLock()
    defer t.mu.RUnlock()
    for _, ch := range t.subs {
        select {
        case ch <- v:
        default:
        }
    }
}
```

For cross-process, use NATS, Kafka, Redis pub/sub, or Google Pub/Sub.

21. Chain of responsibility

A list of handlers; each decides whether to process or pass on.

In Go, this is usually expressed as middleware (handlers chain) or a slice of handlers iterated until one returns true.

```go
type Handler func(req Request) (handled bool, err error)

func ChainOfResponsibility(handlers []Handler) Handler {
    return func(req Request) (bool, error) {
        for _, h := range handlers {
            ok, err := h(req)
            if err != nil { return false, err }
            if ok       { return true, nil }
        }
        return false, nil
    }
}
```

HTTP middleware is the same shape, but each link decides whether to call `next` instead of returning a "handled" bool.

22. Embedding (Go's composition)

Not strictly a "pattern" but the Go answer to inheritance.

```go
type Animal struct { Name string }
func (a Animal) Hello() string { return "hi " + a.Name }

type Dog struct {
    Animal               // embedded
    Breed string
}

d := Dog{Animal: Animal{Name: "Rex"}, Breed: "Lab"}
d.Hello()           // "hi Rex" (promoted from Animal)
```

Methods and fields on the embedded type are promoted. The outer type can shadow them by defining its own.

It's not inheritance. The outer type doesn't become the inner type; it has one as an unnamed field. You can't polymorphically pass a `Dog` where an `Animal` is expected (use interfaces for that).

Use embedding for "X is mostly a Y plus some more." Use composition with named fields for "X has a Y."

What Go doesn't need (and you shouldn't write)

Excessive abstraction

Don't define an interface for every struct. Interfaces belong on the consumer side: define when you need a swap point, not preemptively.

Getter/setter ceremony

Go has no implicit accessors. Exported fields are fine when they're just data. Add `GetX` / `SetX` only when there's actual behavior (validation, caching, change tracking).

```
type Point struct { X, Y int }            // fine
```

Don't write `GetX()` and `SetX(int)` for free.

Class-style inheritance

Embedding is not inheritance. If you find yourself building "abstract base classes" with template methods, you're fighting the language.

Reflection-heavy frameworks

Big DI containers, ORMs with reflective magic, "auto-everything" libraries. Go runs them, but you give up type safety, performance, and clarity.

Premature generics

Generics are great when you have real duplication. Don't generic-ize a single function that's used by one type.

Pattern selection guide

Need	Pattern
Optional construction params	Functional options
Stepwise construction with validation	Builder
Swap algorithm at runtime	Strategy (interface) or function value
Add cross-cutting behavior	Decorator / middleware
Match incompatible interfaces	Adapter
Simplify a complex subsystem	Facade
Cache or lazy-load behind same interface	Proxy
Multiple consumers of an event	Observer (channels)
Iterate without exposing internals	Iterator (`iter.Seq`)
Manage lifecycle / state	State machine
One-time initialization	`sync.Once`
Reusable allocations	`sync.Pool`
Data access boundary	Repository

Need	Pattern
Wire up deps in main	Constructor injection
Bounded concurrency	Worker pool / errgroup
Multi-stage transformation	Pipeline
Combine many streams	Fan-in
Spread work across workers	Fan-out
In-process events	Pub/sub with channels
Try handlers in order	Chain of responsibility

Best practices

1. Don't apply patterns by name. Solve the actual problem. If a struct fits, use a struct.
2. Composition over inheritance. Go doesn't have inheritance; embedding is its closest tool, but it's not the same.
3. Functional options for construction. It's the Go default for anything beyond 2-3 params.
4. Define interfaces on the consumer side. The function that takes the interface declares it.
5. Small interfaces. 1-3 methods. Compose them.
6. Accept interfaces, return concrete types. The standard library does it; so should you.
7. Constructor functions (`NewX`) for any non-trivial construction.
8. Channels for communication, mutexes for state. Both are correct Go; pick by intent.
9. `context.Context` everywhere there's I/O or blocking. First parameter, always.
10. Resist DI frameworks until hand-wiring becomes painful.
11. Prefer plain code over clever patterns. Readable Go is short, direct, and boring.

Common gotchas

Gotcha	Cause	Fix
Interface defined preemptively, no second implementation	Over-abstraction	Define when you have a real swap need
Embedding used like inheritance	Misunderstanding	Use interfaces for polymorphism
Functional options without defaults	Caller sees zero values	Set defaults in the constructor
Builder without final error check	Silent failures	Capture error in builder, return on `Build()`
Decorator chain order matters but isn't documented	Confusing behavior	Document order; pick conventions
Singleton blocks tests	Can't swap implementation	Use constructor injection

Gotcha	Cause	Fix
`sync.Pool` used for large objects	Memory bloat	Pool small, short-lived buffers only
Repository interface mirrors DB exactly	Leaky abstraction	Define from caller's needs
Observer subscribers block on full channel	Deadlock	Use buffered or select default
Pipeline never terminates	Forgot to close source	Always close the upstream channel
Worker pool deadlock	Main goroutine sends synchronously, no readers	Buffer results channel or use select with ctx
State machine has invalid transitions	No guard	Explicit checks in each transition method
Generic for one type	Over-engineering	Concrete is fine for one type
Reflection-based "magic"	Slow, fragile	Plain code, codegen, or generics

Quick reference

Pattern	Go idiom
Functional options	`type Option func(*T); func With(...) Option`
Builder	Struct + chained methods + `Build() (T, error)`
Strategy	Interface or `func(...) ...` parameter
Decorator	Wrap interface, add behavior
Middleware	`func(http.Handler) http.Handler`
Adapter	Small wrapper struct (or `http.HandlerFunc` trick)
Facade	A struct with methods that hide internals
Proxy	Same shape as decorator
Observer	Channels + Subscribe/Unsubscribe
Iterator	`iter.Seq[T]`, `iter.Seq2[K,V]` (Go 1.23+)
State machine	Explicit state field + method guards
Singleton	`sync.Once` + package var
Object pool	`sync.Pool`
Repository	Interface in domain, impl in adapter
DI	Constructor injection in main
Worker pool	Channel + goroutines + WaitGroup (or errgroup)
Pipeline	Channel-connected stages

Pattern	Go idiom
Fan-in	Read N channels, write 1
Fan-out	N goroutines read 1 channel
Pub/sub	Topic with map of subscribers
Chain of responsibility	Iterate handlers until handled
Embedding	Unnamed struct field
Composition	Named struct field